

Instituto Tecnológico de Costa Rica

Escuela de Computación

Lenguajes de programación

Proyecto Programado #2

Sistema de eventos comerciales

Código de curso: IC4700

Estudiante:

Alice Arias Salazar | 2023104639

Profesor:

Allan Rodríguez Davila

Grupo: 60

I Semestre 2026

Índice

Manual de usuario y pruebas de funcionalidad 3

1. Clonar el repositorio 3

2. Abrir el proyecto en Visual Studio Code 3

3. Ejecutar el programa 3

4. Menú principal 4

Transformación de eventos 6

1. Aplicar impuestos + etiquetas 8

2. Solo impuestos 9

3. Solo etiquetas 11

Análisis de datos 12

1. Monto total 13

2. Promedio por categoría por año 14

Análisis temporal 15

1. Mes y día más activo 16

2. Eventos extremos 17

3. Resumen por intervalo de días 18

Búsqueda 19

Estadísticas 21

Salir 22

Descripción del problema 24

Transformación de eventos 25

Análisis de datos 25

Análisis temporal 25

Búsqueda por rangos de fechas 25

Generación de estadísticas 26

Diseño del programa: 26

Algoritmos más importantes creados 30

1. Algoritmo de cálculo de impuestos 30

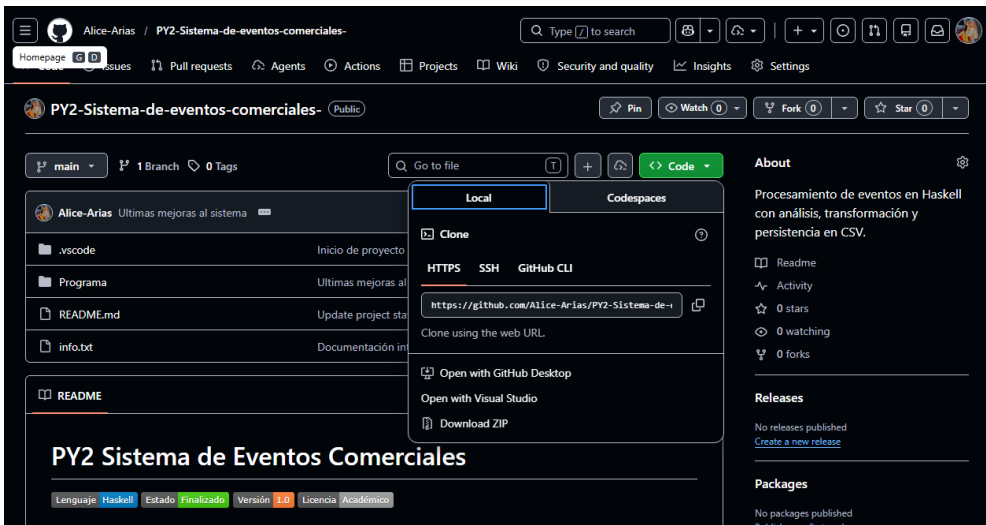
2. Algoritmo de promedio de compras.....	31
3. Algoritmo de etiquetado de eventos por valor.....	31
4. Algoritmo de análisis del mes con mayor ingreso	31
5. Algoritmo de día con mayor actividad	31
6. Algoritmo de eventos extremos	32
7. Algoritmo de agrupación por intervalos de tiempo	32
8. Algoritmo de análisis de rango de fechas.....	32
9. Algoritmo de cálculo de totales por evento	32
10. Algoritmo de lectura y escritura de CSV	32
Librerías usadas:.....	33
Análisis de resultados	36
Objetivos alcanzados	36
Justificación de toma de decisiones	38
Map.....	38
Filter	38
Any	38
maximumBy y minimumBy.....	39
mapM y mapM_.....	39
group y groupBy	39
sort, sortBy y sortOn	40
lookup	40
Encadenamiento de funciones.....	40
Bitácora	41

Manual de usuario y pruebas de funcionalidad

1. Clonar el repositorio

Primero deben clonar el repositorio desde GitHub usando GitHub Desktop: <https://github.com/Alice-Arias/Proyecto1-Lenguajes-Gestion-de-Ventas.git> Esto descargará todos los archivos en una carpeta local en su computadora.

Figura 1



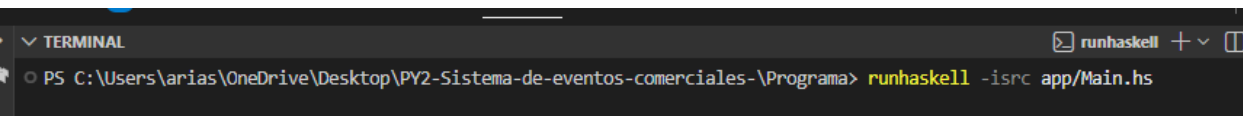
Si prefieres hacerlo de otra manera, puedes ingresar al repositorio y hacer clic en Code. Luego, como se muestra en la Figura 1, selecciona la opción Download ZIP. Una vez que la descarga se complete, descomprime el archivo y estarás listo para continuar con el siguiente paso.

2. Abrir el proyecto en Visual Studio Code

Una vez descargado, abran Visual Studio Code y carguen la carpeta del proyecto vamos a abrir una terminal nueva esto para comenzar a compilar el proyecto.

3. Ejecutar el programa

Figura 2



Al abrirla, es importante verificar que la ruta actual corresponda a la carpeta del proyecto, ya que el comando de ejecución solo funcionará correctamente si se está ubicado en el directorio adecuado se debe ver como la figura 2.

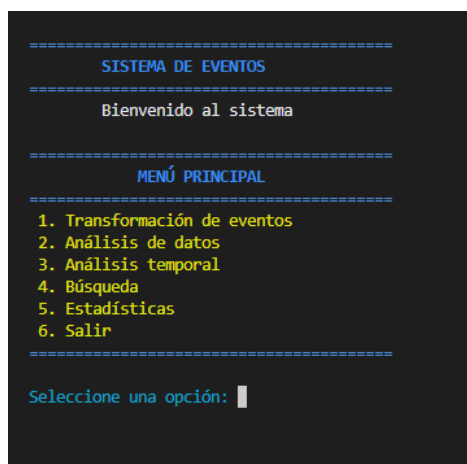
Una vez dentro de la ruta correcta, se procede a ejecutar el programa utilizando el comando **runhaskell -isrc app/Main.hs**. Este comando permite ejecutar directamente el archivo principal del sistema sin necesidad de compilarlo previamente. La opción **runhaskell** indica que se va a ejecutar código Haskell, mientras que **-isrc** le indica al sistema que los módulos del proyecto se encuentran dentro de la carpeta **src**, lo cual es necesario para que pueda resolver correctamente las dependencias, **app/Main.hs** representa el archivo principal desde donde inicia la ejecución del programa.

Al ejecutar el comando, el sistema se inicia automáticamente y se muestra el menú principal en la consola que se mostrara en la siguiente opción.

En caso de que el programa no se ejecute correctamente, es importante verificar que la terminal esté ubicada dentro de la carpeta correcta del proyecto, que Haskell esté instalado en el sistema y que el comando haya sido escrito de forma exacta. Con estos pasos correctamente realizados, el sistema debería ejecutarse sin inconvenientes.

4. Menú principal

Figura 3



```
=====
SISTEMA DE EVENTOS
=====
Bienvenido al sistema

=====
MENÚ PRINCIPAL
=====
1. Transformación de eventos
2. Análisis de datos
3. Análisis temporal
4. Búsqueda
5. Estadísticas
6. Salir
=====
Seleccione una opción: |
```

Al ejecutar el sistema, lo primero que se muestra es la pantalla de bienvenida, donde se presenta el nombre del sistema y un mensaje de introducción. Luego aparece el menú principal, que es el punto central desde el cual el usuario puede interactuar con todas las funciones disponibles del programa como se muestra en la figura 3.

Nota importante: el archivo eventos.csv **nunca se borra ni se reinicia**, sino que funciona como un registro acumulativo del sistema.

El menú principal está organizado en seis opciones, cada una diseñada para cumplir una función específica dentro del sistema de eventos.

- La primera opción corresponde a la “Transformación de eventos”, la cual se encarga de modificar o actualizar los datos de los eventos, aplicando cálculos como impuestos, totales o ajustes internos del sistema. Esta opción es fundamental porque garantiza que la información esté correctamente procesada antes de ser analizada.
- La segunda opción es el “Análisis de datos”, donde el sistema procesa la información de los eventos para generar resultados generales. Aquí se pueden obtener métricas como totales, promedios y agrupaciones por categoría, lo que permite entender el comportamiento general de los eventos registrados.
- La tercera opción corresponde al “Análisis temporal”, que se enfoca en el comportamiento de los eventos a lo largo del tiempo. En esta sección se pueden observar fechas más activas, rangos de tiempo, eventos más antiguos y recientes, lo cual permite analizar cómo varía la actividad en diferentes periodos.
- La cuarta opción es la “Búsqueda”, la cual permite al usuario consultar eventos dentro de un rango de fechas específico. Esta función es útil para filtrar información concreta y visualizar únicamente los eventos que ocurrieron dentro de un periodo determinado.
- La quinta opción corresponde a “Estadísticas”, donde se generan reportes más completos del sistema. Aquí se incluyen resúmenes por categoría,

eventos con mayor y menor valor, y análisis generales que incluso pueden guardarse en archivos externos como CSV para su posterior uso.

- La sexta opción es “Salir”, la cual permite cerrar el sistema de manera ordenada. Al seleccionar esta opción, el programa finaliza su ejecución y regresa a la consola.

Cada una de estas opciones será explicada de forma detallada en las siguientes secciones, donde se mostrará su funcionamiento, su propósito dentro del sistema y cómo el usuario puede utilizarlas correctamente.

Transformación de eventos

Figura 4

```
=====
ACTUALIZACIÓN SISTEMA
=====
Eventos agregados: 23
=====

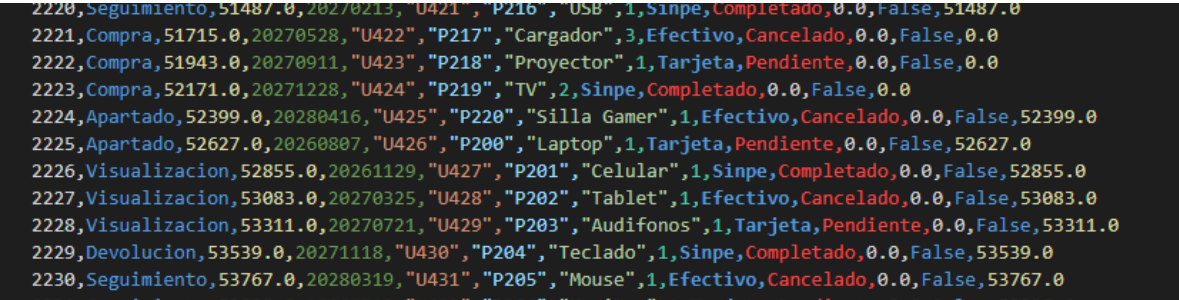
=====
TRANSFORMACIÓN DE EVENTOS
=====
1. Aplicar impuestos + etiquetas
2. Solo impuestos
3. Solo etiquetas
=====
Seleccione una opción:
|
```

Al seleccionar la opción 1 del menú principal, el sistema muestra un mensaje de “Actualización del sistema” junto con la cantidad de eventos agregados. Este mensaje no es solo informativo, sino que representa la creación y carga de nuevos eventos dentro del sistema como se muestra en la figura 4. En otras palabras, en este punto el sistema está generando nuevos registros que serán utilizados para todo el análisis posterior, trabajamos con un archivo que tiene datos de otras ejecuciones.

Estos eventos generados se guardan en un archivo llamado eventos.csv, el cual actúa como la base de datos principal del sistema. Cada vez que se actualiza el sistema, se agregan nuevos registros a este archivo, manteniendo así un historial

continuo de eventos. Estos registros se ven de forma estructurada como se muestra en la figura 5.

Figura 5



Cada línea representa un evento completo del sistema, y todos siguen la estructura definida por el tipo de dato Evento. Este tipo de dato está compuesto por varios campos que describen toda la información del evento. Incluye el identificador del evento único, la categoría, el valor, la fecha en formato numérico, el usuario, el producto, la cantidad, el método de pago, el estado, el impuesto, una etiqueta y el total final.

En este modelo, todos los eventos se crean inicialmente con ciertos valores por defecto. Por ejemplo, cuando el evento es una compra, el campo de impuesto y total inicia en 0, al igual que el campo de etiqueta, que se establece en False. Esto significa que en ese momento el evento aún no ha sido procesado completamente. Estos valores no son finales, ya que el sistema trabaja bajo un flujo dinámico donde los datos pueden ser modificados posteriormente.

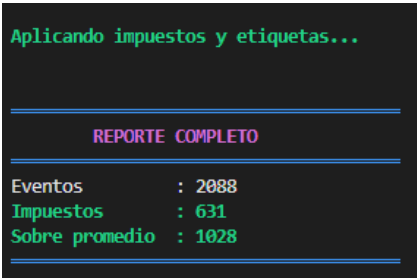
La transformación de estos valores ocurre únicamente cuando el usuario accede a las opciones del menú de transformación. Es en ese momento cuando el sistema aplica reglas de negocio como el cálculo de impuestos, la actualización de totales y la asignación de etiquetas según el comportamiento del evento. Por eso, los eventos que se generan inicialmente se consideran “eventos base” o “eventos sin procesar” de igual manera se puede trabajar con esto sin ningún problema, pero para el documento se debe realizar la transformación.

Además, es importante entender que el sistema trabaja con eventos actualizados de forma constante. Cada vez que el usuario entra a una opción del menú principal, el sistema puede generar o actualizar eventos nuevos, lo que da la sensación de un flujo dinámico de información. Esto permite simular un entorno real donde los datos están en constante cambio y no son estáticos.

Más adelante, se explicará cómo estos valores iniciales como el impuesto en 0 o la etiqueta en False son modificados según reglas específicas del sistema, dando como resultado eventos completamente procesados y listos para análisis.

1. Aplicar impuestos + etiquetas

Figura 6



Al seleccionar la opción “Aplicar impuestos + etiquetas” dentro del menú de transformación, el sistema realiza un procesamiento completo de los eventos como se muestra en la figura 6. En este punto, los eventos que antes estaban “sin procesar” pasan a ser eventos actualizados, lo que significa que se les aplican tanto los cálculos de impuestos como la evaluación de etiquetas según su comportamiento como se muestra en la figura 7.

Figura 7



Cuando el sistema indica “Aplicando impuestos y etiquetas...”, lo que está ocurriendo internamente es que se recorren todos los eventos registrados y se modifican ciertos campos clave debemos aclarar que esto solo se hace en los eventos que no se han calculado. En el caso de los eventos de tipo Compra, se calcula un impuesto basado en el valor del evento y la cantidad, y ese impuesto se guarda en el campo correspondiente. Además, el total del evento se actualiza sumando el subtotal más el impuesto, lo que explica por qué antes aparecía en 0 y ahora tiene un valor completo.

Por otro lado, el campo etiqueta cambia de False a True en aquellos eventos cuyo valor supera el promedio de su categoría. Esto significa que el sistema analiza todos los eventos, agrupa por categoría (por ejemplo, Compra, Visualización, etc.), calcula un promedio por cada grupo y luego marca con True los eventos que están por encima de ese promedio. Esta es una forma de identificar eventos relevantes o de alto valor dentro del sistema.

Un aspecto importante es que estos cambios no son temporales. Una vez que se aplica la transformación, los datos se actualizan directamente en el archivo eventos.csv. Es decir, si se revisan los registros después de ejecutar esta opción, ya se podrán ver los impuestos calculados, los totales corregidos y las etiquetas en True, como en los ejemplos mostrados. Esto confirma que el sistema no solo analiza, sino que también persiste los resultados.

El sistema muestra un reporte completo como se muestre en la figura 6, donde se resume el estado actual de los eventos. Aquí se indica la cantidad total de eventos procesados, cuántos eventos de tipo compra tienen impuesto aplicado y cuántos eventos están por encima del promedio (es decir, cuántos tienen la etiqueta en True). Este reporte permite tener una visión rápida del impacto de la transformación realizada.

2. Solo impuestos

Figura 8

```
Aplicando impuestos...

=====
                REPORTE DE IMPUESTOS
=====
Total eventos   : 2099
Compras         : 634
Con impuesto    : 634
=====
```

Al seleccionar la opción “Solo impuestos” dentro del menú de transformación, el sistema realiza un proceso más específico: únicamente calcula y aplica los impuestos a los eventos correspondientes, sin modificar las etiquetas. A diferencia de la opción anterior, aquí no se evalúa si el evento está por encima del promedio, sino que se enfoca únicamente en completar el cálculo monetario de las compras.

Cuando aparece el mensaje “Aplicando impuestos...”, el sistema lo que hace es revisar el archivo eventos.csv, tomar todos los eventos almacenados y aplicar el cálculo únicamente donde corresponde. Es decir, identifica los eventos de categoría Compra y, sobre esos, calcula el impuesto según su valor y cantidad, actualizando luego el total final. Los demás eventos no se ven afectados, ya que no generan impuesto dentro del sistema.

Un punto importante es entender cómo funciona el flujo del sistema. Cada vez que se selecciona una opción del menú, el sistema procesa la información, muestra su respectivo reporte y luego regresa automáticamente al menú principal. Cuando el usuario vuelve a ingresar a una opción nuevamente, el sistema genera nuevos eventos antes de realizar cualquier procesamiento, lo que hace que la cantidad total de eventos vaya aumentando constantemente.

A pesar de este crecimiento continuo, el sistema evita errores en los cálculos. El impuesto solo se aplica a los eventos que aún no lo tienen calculado (es decir, aquellos con impuesto en 0 en la categoría Compra). Si un evento ya fue procesado anteriormente, el sistema lo deja intacto. Esto garantiza que los valores no se dupliquen ni se alteren incorrectamente con cada ejecución.

Se muestra el reporte de impuestos, donde se resumen tres datos clave: el total de eventos registrados, la cantidad de eventos de tipo compra y cuántos de estos ya tienen impuesto aplicado como se ve en la figura 8.

3. Solo etiquetas

Figura 9

Aplicando etiquetas...

REPORTE DE ETIQUETAS			
Total eventos	:	2112	
Sobre promedio	:	1051	
Categoría	Total	Promedio	Sobre promedio
Visualizacion	420	CRC 36,619.43	211
Devolucion	301	CRC 36,791.82	144
Seguimiento	325	CRC 36,685.78	166
Compra	637	CRC 36,312.12	319
Apartado	429	CRC 37,153.32	211

Al seleccionar la opción “Solo etiquetas” dentro del menú de transformación, el sistema realiza un proceso enfocado únicamente en el análisis de los eventos para determinar cuáles están por encima del promedio dentro de su categoría. A diferencia de las otras opciones, aquí no se calculan impuestos ni se modifican valores monetarios; el objetivo es identificar eventos relevantes según su comportamiento.

Cuando aparece el mensaje “Aplicando etiquetas...”, el sistema revisa el archivo eventos.csv, toma todos los eventos almacenados y los agrupa por categoría, como por ejemplo Visualización, Compra, Devolución, entre otros. Luego calcula el promedio de valor para cada categoría y compara cada evento contra ese promedio. Si el valor del evento es mayor al promedio de su grupo, se le asigna la etiqueta True; en caso contrario, se mantiene o se deja en False.

Al igual que en las otras opciones, el sistema sigue un flujo dinámico. Cada vez que se ejecuta una opción del menú, se muestra el reporte correspondiente y luego se regresa al menú principal. Cuando el usuario vuelve a ingresar, se generan

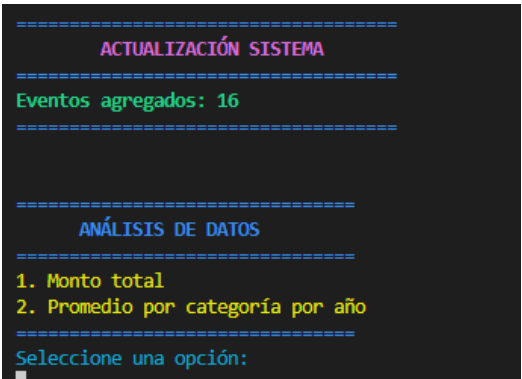
nuevos eventos, lo que provoca que los promedios cambien y que el análisis de etiquetas se vuelva a realizar con información actualizada.

Es importante entender que este proceso no es estático. Las etiquetas pueden cambiar dependiendo del comportamiento de los datos. Un evento que antes no estaba sobre el promedio podría estarlo después, o viceversa, ya que el promedio de cada categoría se recalcula con todos los eventos actuales del sistema.

Se muestra el reporte de etiquetas, donde se presenta un resumen completo. Primero se indica el total de eventos y cuántos están sobre el promedio. Luego se muestra una tabla por categoría, donde se incluye la cantidad de eventos, el promedio calculado y cuántos eventos superan ese promedio como se ve en la figura 9.

Análisis de datos

Figura 10



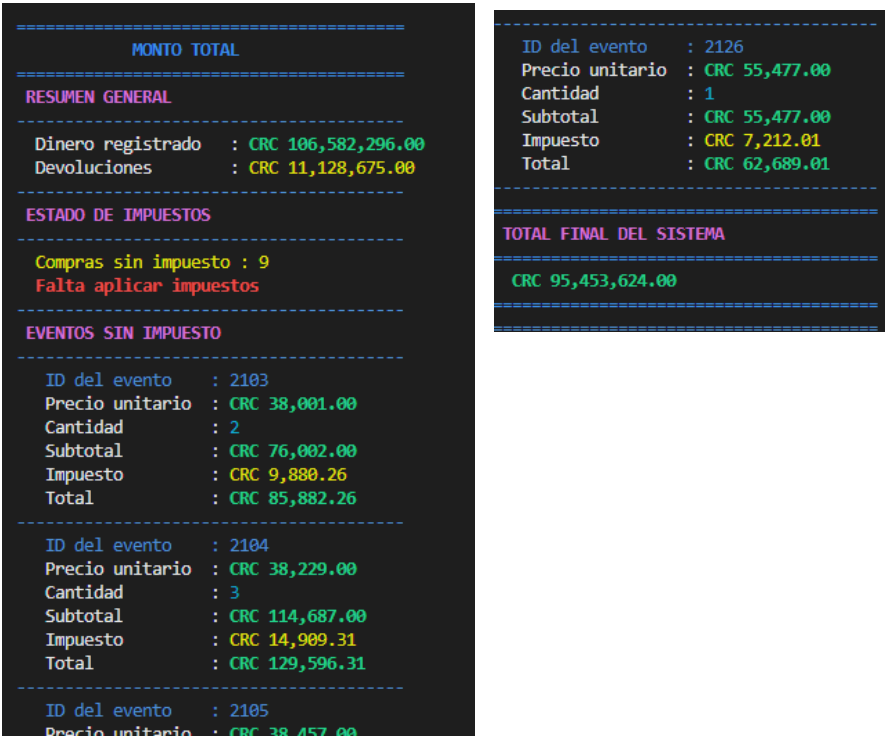
Al seleccionar la opción “Análisis de datos” desde el menú principal, el sistema entra en una sección dedicada a procesar la información de los eventos desde un enfoque numérico y resumido. Antes de mostrar las opciones disponibles, el sistema vuelve a realizar una actualización, generando nuevos eventos que se agregan al archivo eventos.csv. Esto mantiene el comportamiento dinámico del sistema, donde los datos crecen constantemente y el análisis siempre se hace sobre información actualizada.

Una vez completada la actualización, se presenta el menú de análisis de datos. Este menú contiene opciones como se muestra en la figura 10enfocadas en

obtener datos generales que ayudan a entender el comportamiento económico y estadístico de los eventos registrados. A diferencia del módulo de transformación, aquí no se modifican los datos, sino que se interpretan y se calculan resultados a partir de ellos, cada opción se explica a continuación.

1. Monto total

Figura 11



Al seleccionar la opción “Monto total” dentro del módulo de análisis de datos, el sistema realiza un cálculo general de todo el dinero que ha pasado por los eventos registrados. Antes de mostrar el resultado, como ya es habitual, el sistema puede haber agregado nuevos eventos al archivo eventos.csv, por lo que el análisis siempre se hace con información actualizada.

En esta sección se muestra primero un resumen general, donde se presenta el total de dinero registrado en el sistema. Este valor corresponde a la suma de todos los eventos, considerando sus montos finales. Además, se muestra el total de **devoluciones**, que en este sistema se interpretan como pérdidas. Por eso, aunque

están incluidas en los registros, se separan visualmente, ya que luego se toman en cuenta para ajustar el total real del sistema como se muestra en la figura 11.

Un punto clave de esta opción es que el sistema también revisa si existen **eventos de compra que aún no tienen impuesto aplicado**. En lugar de ignorarlos o modificar directamente el archivo, lo que hace el sistema es identificarlos y mostrarlos en pantalla. Para cada uno, se presenta el detalle completo: precio unitario, cantidad, subtotal, el impuesto que debería aplicarse y el total correcto. Aunque esos eventos no tengan el impuesto aplicado en el archivo eventos.csv, el sistema **los calcula en tiempo real sin modificar el archivo**. Esto permite obtener un monto total correcto y actualizado sin alterar los datos originales. Es decir, el sistema simula el cálculo faltante solo para efectos del análisis.

Se garantiza que el **total final del sistema** siempre sea preciso. Primero se suma todo el dinero registrado, luego se consideran las devoluciones como pérdidas, y además se ajustan los eventos que aún no tienen impuesto aplicado. Todo esto se hace sin comprometer la integridad del archivo original.

2. Promedio por categoría por año

Figura 12

PROMEDIO POR CATEGORÍA POR AÑO	
AÑO: 2026	
CATEGORÍA	PROMEDIO
Visualizacion	CRC 38,292.50
Devolucion	CRC -38,396.73
Seguimiento	CRC 34,763.45
Compra	CRC 83,482.68
Apartado	CRC 36,448.99
AÑO: 2027	
CATEGORÍA	PROMEDIO
Visualizacion	CRC 36,911.41
Devolucion	CRC -34,210.67
Seguimiento	CRC 38,558.25
Compra	CRC 78,766.69
Apartado	CRC 36,951.14
AÑO: 2028	
CATEGORÍA	PROMEDIO
Visualizacion	CRC 34,499.26
Devolucion	CRC -44,209.83
Seguimiento	CRC 37,526.35
Compra	CRC 74,715.04
Apartado	CRC 40,360.09

Al seleccionar la opción “Promedio por categoría por año”, el sistema realiza un análisis más detallado de los eventos, organizando la información según el año en que ocurrieron y la categoría a la que pertenecen. Este proceso no modifica ningún dato del archivo eventos.csv, sino que utiliza la información existente.

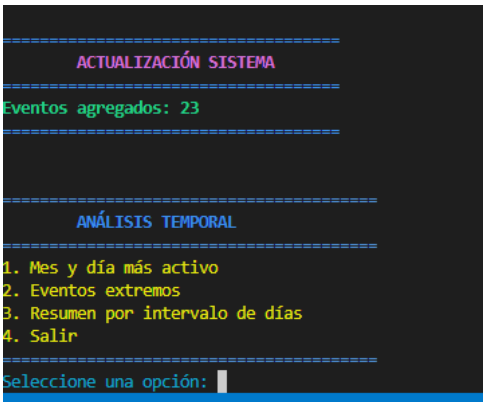
Primero, el sistema agrupa todos los eventos por año (por ejemplo, 2026, 2027, 2028). Dentro de cada año, los eventos se vuelven a agrupar por categoría, como Visualización, Compra, Devolución, Seguimiento y Apartado. Una vez organizados de esta manera, se calcula el promedio del valor de los eventos en cada categoría para ese año específico.

El resultado se muestra en forma de tabla como se ve en la figura 12, donde se puede observar el promedio por categoría en cada año. Esto permite identificar patrones importantes, como qué categorías manejan montos más altos o cómo cambian los valores con el tiempo. Por ejemplo, la categoría **Compra** suele tener promedios más altos porque incluye el total de las transacciones, mientras que **Visualización** o **Seguimiento** manejan valores más bajos.

Un detalle importante es el caso de las **devoluciones**, cuyos valores aparecen en negativo. Esto se debe a que el sistema las considera como pérdidas, por lo que afectan directamente el promedio de su categoría. De esta forma, el análisis refleja de manera más realista el impacto económico de este tipo de eventos.

Análisis temporal

Figura 13



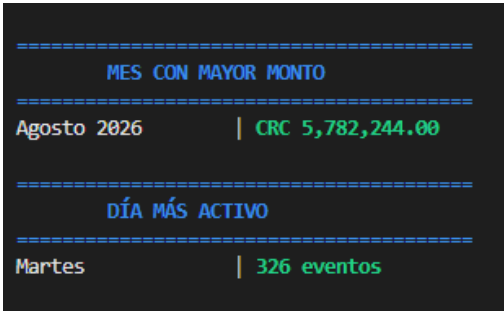
Al seleccionar la opción “Análisis temporal” desde el menú principal, el sistema permite estudiar el comportamiento de los eventos a lo largo del tiempo. Antes de mostrar las opciones disponibles, se realiza nuevamente una actualización del sistema, donde se generan nuevos eventos que se agregan al archivo eventos.csv. Esto mantiene el flujo dinámico del sistema, asegurando que el análisis se realice siempre con datos recientes.

Una vez completada la actualización, se presenta el menú de análisis temporal, el cual incluye varias opciones como se muestra en la figura 13, enfocadas en examinar cómo se distribuyen los eventos en el tiempo. Este módulo no modifica los datos, sino que los organiza y analiza para ofrecer información útil sobre fechas, frecuencia y patrones de actividad.

Las opciones disponibles permiten identificar aspectos clave como el mes y el día con mayor actividad, los eventos más antiguos y recientes, y resúmenes agrupados por intervalos de días que se explican a continuación.

1. Mes y día más activo

Figura 14



Al seleccionar la opción “Mes y día más activo”, el sistema realiza un análisis del comportamiento temporal de los eventos para identificar en qué momento se concentra la mayor actividad. Este análisis se basa en todos los eventos registrados en el archivo eventos.csv, incluyendo los más recientes generados durante la actualización del sistema.

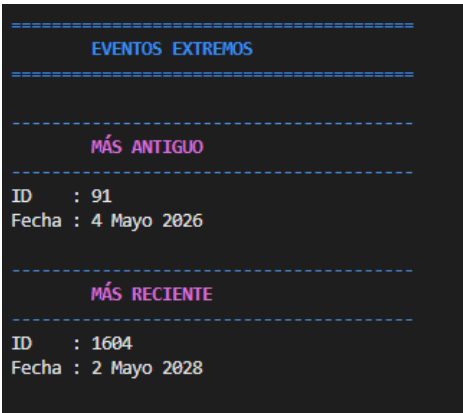
En la primera parte, el sistema muestra el mes con mayor monto, que corresponde al período en el que se ha acumulado la mayor cantidad de dinero a

partir de los eventos. Para obtener este resultado, se agrupan los eventos por mes y año, se suman sus montos y luego se identifica cuál de esos grupos tiene el valor total más alto como se muestra en la figura 14. Esto permite reconocer en qué momento el sistema ha tenido mayor movimiento económico.

En la segunda parte, se presenta el día más activo, que indica cuál día de la semana concentra la mayor cantidad de eventos registrados. Para calcularlo, el sistema analiza las fechas de todos los eventos, identifica el día correspondiente (por ejemplo, lunes, martes, etc.) y cuenta cuántos eventos ocurrieron en cada uno. El día con mayor cantidad es el que se muestra como resultado como se muestra en la figura 14.

2. Eventos extremos

Figura 15



Al seleccionar la opción “Eventos extremos”, el sistema muestra cuáles son los eventos más antiguos y recientes registrados en el archivo eventos.csv. Este análisis permite ubicar los límites del rango temporal con el que está trabajando el sistema, es decir, desde cuándo hay registros hasta la fecha más actual disponible.

Para obtener esta información, el sistema revisa todos los eventos almacenados y los ordena según su fecha. A partir de ese orden, identifica el evento con la fecha más antigua y el evento con la fecha más reciente. Estos dos valores se presentan en pantalla junto con su identificador y la fecha en un formato más legible.

El evento más antiguo representa el primer registro dentro del sistema, lo que permite entender desde qué momento se comenzó a almacenar información. Por otro lado, el evento más reciente indica el último registro generado, lo cual refleja hasta qué punto llegan los datos actuales, incluyendo los eventos nuevos que se agregan en cada actualización como se muestra en la figura 15.

3. Resumen por intervalo de días

Figura 16

Ingrese el intervalo en días (1 - 729): 100

RESUMEN POR INTERVALO DE DÍAS		
Rango de Fechas	Cantidad	Monto Total
4 Mayo 2026 - 11 Agosto 2026	264	CRC 13,600,416.00
12 Agosto 2026 - 19 Noviembre 2026	347	CRC 17,648,678.00
20 Noviembre 2026 - 27 Febrero 2027	297	CRC 14,784,289.00
28 Febrero 2027 - 7 Junio 2027	286	CRC 14,093,688.00
8 Junio 2027 - 15 Septiembre 2027	269	CRC 12,791,255.00
16 Septiembre 2027 - 24 Diciembre 2027	310	CRC 16,175,197.00
25 Diciembre 2027 - 2 Abril 2028	297	CRC 14,384,123.00
3 Abril 2028 - 2 Mayo 2028	100	CRC 4,801,790.50

Al seleccionar la opción “Resumen por intervalo de días”, el sistema permite analizar cómo se distribuyen los eventos en bloques de tiempo definidos por el usuario. En este caso, el usuario debe ingresar un intervalo de días que servirá para agrupar los eventos y generar un resumen más organizado de la información.

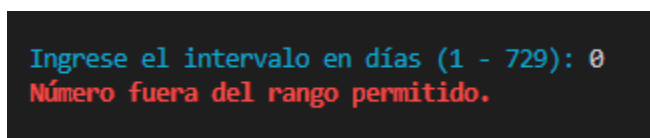
Cuando el sistema solicita “Ingrese el intervalo en días”, se debe escribir un número dentro del rango permitido (por ejemplo, entre 1 y 729 el 729 puede cambiar dependiendo los eventos que tengamos). Este valor indica el tamaño de cada bloque de tiempo. Por ejemplo, si se ingresa 100, el sistema agrupará los eventos en períodos de 100 días consecutivos como se muestra en la figura 16.

Una vez ingresado un valor válido, el sistema toma todos los eventos del archivo eventos.csv, los ordena por fecha y los divide en intervalos según el número indicado. Luego, para cada intervalo, calcula dos datos principales: la cantidad de eventos ocurridos en ese rango de fechas y el monto total generado en ese mismo período. Estos resultados se muestran en forma de tabla, donde cada fila representa

un intervalo con su respectivo rango de fechas, cantidad de eventos y monto acumulado.

Es importante tener en cuenta que el sistema valida la entrada del usuario. Si se ingresa un valor fuera del rango permitido, como por ejemplo 0, el sistema mostrará un mensaje indicando que el número está fuera del rango y volverá a solicitar el dato. Esto garantiza que el programa funcione correctamente y evita errores en el procesamiento como se muestra en la figura 17.

Figura 17



Búsqueda

Al seleccionar la opción “Búsqueda” en el menú principal, el sistema permite consultar eventos específicos dentro de un rango de fechas definido por el usuario. Como en los demás módulos, antes de iniciar la búsqueda se realiza una actualización del sistema, donde se generan nuevos eventos que se agregan al archivo eventos.csv, asegurando que la consulta se haga con datos recientes.

El sistema solicita primero la fecha de inicio y luego la fecha final, ambas en formato dd-mm-yyyy. Es importante ingresar las fechas correctamente, ya que si el formato es inválido (por ejemplo, escribir solo “1”), el sistema mostrará un mensaje de error y volverá a pedir la fecha. Esto garantiza que la búsqueda se realice correctamente y evita problemas en el procesamiento.

Figura 18

BÚSQUEDA POR RANGO DE FECHAS

Ingrese fecha de inicio:
10-10-2026
Ingrese fecha final:
15-10-2026

BÚSQUEDA POR RANGO DE FECHAS

Desde: 10-10-2026
Hasta: 15-10-2026

FECHA	ID	CATEGORIA	VALOR	CANT	TOTAL
10 Octubre 2	358	Seguimiento	CRC 7,452.00	1	CRC 7,452.00
10 Octubre 2	1175	Devolucion	CRC 67,878.00	1	CRC 67,878.00
10 Octubre 2	1370	Visualizacion	CRC 62,151.00	1	CRC 62,151.00
11 Octubre 2	436	Compra	CRC 19,161.00	3	CRC 64,955.79
11 Octubre 2	1298	Apartado	CRC 10,765.00	1	CRC 10,765.00
11 Octubre 2	1648	Compra	CRC 59,550.00	3	CRC 201,874.50
11 Octubre 2	1695	Apartado	CRC 45,654.00	1	CRC 45,654.00
13 Octubre 2	107	Seguimiento	CRC 48,435.00	1	CRC 48,435.00
13 Octubre 2	726	Devolucion	CRC 26,799.00	1	CRC 26,799.00
13 Octubre 2	1674	Visualizacion	CRC 42,221.00	1	CRC 42,221.00
13 Octubre 2	1771	Visualizacion	CRC 45,118.00	1	CRC 45,118.00
13 Octubre 2	1797	Compra	CRC 30,900.00	2	CRC 69,834.00
13 Octubre 2	2006	Compra	CRC 62,445.00	1	CRC 70,562.85
15 Octubre 2	78	Compra	CRC 67,702.00	2	CRC 153,006.52
15 Octubre 2	1081	Compra	CRC 60,784.00	3	CRC 206,057.77
15 Octubre 2	1777	Seguimiento	CRC 9,469.00	1	CRC 9,469.00

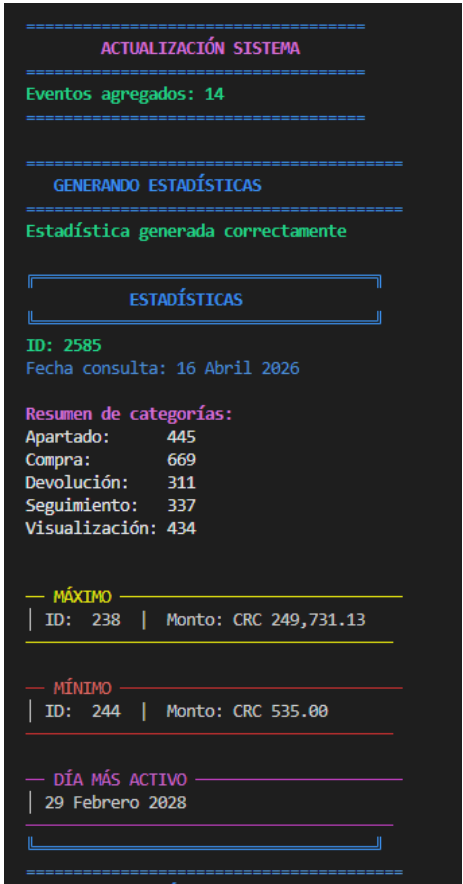
Total encontrados: 16
Seguimiento : CRC 65,356.00
Devolucion : CRC -94,677.00
Visualizacion : CRC 149,490.00
Compra : CRC 766,291.40
Apartado : CRC 56,419.00
Monto final de ganancias : CRC 942,879.40

Una vez ingresadas ambas fechas de forma válida, el sistema filtra todos los eventos que se encuentran dentro de ese rango. Luego, los ordena por fecha y los muestra en forma de tabla. En esta tabla se incluye información relevante de cada evento, como la fecha, el ID, la categoría, el valor, la cantidad y el total. Esto permite al usuario visualizar de manera clara y organizada todos los eventos que ocurrieron en ese período específico como se ve en la figura 18.

Después de mostrar el detalle de los eventos, el sistema presenta un resumen general. En esta sección se indica cuántos eventos fueron encontrados y se desglosa el monto total por cada categoría. Aquí es importante notar que las devoluciones se manejan como valores negativos, ya que representan pérdidas dentro del sistema como se ve en la figura 18. Se muestra el monto final de ganancias, el cual se obtiene sumando todos los valores y restando automáticamente las devoluciones.

Estadísticas

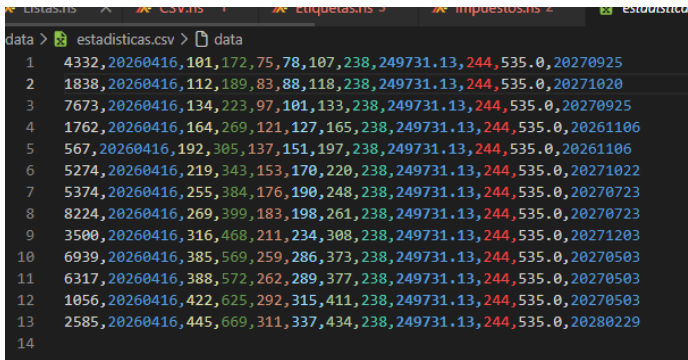
Figura 19



Al seleccionar la opción “Estadísticas”, el sistema genera un reporte completo que resume el estado general de todos los eventos registrados. Como ya es parte del funcionamiento normal, antes de generar este reporte se realiza una actualización del sistema, agregando nuevos eventos al archivo eventos.csv. Esto asegura que las estadísticas siempre se calculen con la información más reciente.

Una vez actualizados los datos, el sistema comienza el proceso de generación de estadísticas. Cuando aparece el mensaje “Estadística generada correctamente”, significa que toda la información fue procesada y además se ha guardado un registro en un archivo externo estadísticas csv como se ve en la figura 20, permitiendo conservar un historial de consultas realizadas.

Figura 20



El reporte inicia mostrando un identificador único de la estadística generada, junto con la fecha de consulta, lo que permite diferenciar cada análisis realizado en el sistema. Luego se presenta un resumen por categorías, donde se indica cuántos eventos existen en cada tipo, como Apartado, Compra, Devolución, Seguimiento y Visualización. Esto da una visión clara de la distribución de los eventos dentro del sistema.

A continuación, se muestran tres indicadores clave. El primero es el evento con mayor monto, donde se identifica el evento que tiene el valor más alto registrado, incluyendo su ID y el monto correspondiente. Luego se presenta el evento con menor monto, que permite conocer el valor más bajo dentro del sistema. Estos dos datos ayudan a entender los extremos económicos de los eventos, se muestra el día más activo, que corresponde a la fecha en la que se registró la mayor cantidad de eventos como se ve en la figura 19.

Salir

Figura 21

```
2086,Seguimiento,36475.0,20260817,"U587","P208","Camara",1,Efectivo,Cancelado,0.0,False,36475.0
2087,Seguimiento,36703.0,20280305,"U588","P209","Consola",1,Tarjeta,Pendiente,0.0,True,36703.0
2088,Visualizacion,44030.0,20270929,"U589","P210","Smartwatch",1,Sinpe,Completado,0.0,True,44030.0
2089,Devolucion,44258.0,20270422,"U590","P211","Bocina",1,Efectivo,Cancelado,0.0,True,44258.0
2090,Devolucion,44486.0,20261115,"U591","P212","Microfono",1,Tarjeta,Pendiente,0.0,True,44486.0
2091,Seguimiento,44714.0,20260612,"U592","P213","Router",1,Sinpe,Completado,0.0,True,44714.0
2092,Compra,44942.0,20280109,"U593","P214","DiscoDuro",3,Efectivo,Cancelado,17527.38,True,152353.38
2093,Compra,45170.0,20270810,"U594","P215","SSD",1,Tarjeta,Pendiente,5872.1,True,51042.1
2094,Compra,45398.0,20270313,"U595","P216","USB",2,Sinpe,Completado,11803.48,True,102599.48
2095,Apartado,45626.0,20261016,"U596","P217","Cargador",1,Efectivo,Cancelado,0.0,True,45626.0
2096,Apartado,45854.0,20260523,"U597","P218","Proyector",1,Tarjeta,Pendiente,0.0,True,45854.0
2097,Visualizacion,46082.0,20271230,"U598","P219","TV",1,Sinpe,Completado,0.0,True,46082.0
2098,Devolucion,46310.0,20270810,"U599","P220","Silla Gamer",1,Efectivo,Cancelado,0.0,True,46310.0
2099,Visualizacion,37089.0,20270509,"U300","P200","Laptop",1,Tarjeta,Pendiente,0.0,True,37089.0
2100,Devolucion,37317.0,20261222,"U301","P201","Celular",1,Sinpe,Completado,0.0,True,37317.0
2101,Seguimiento,37545.0,20260808,"U302","P202","Tablet",1,Efectivo,Cancelado,0.0,True,37545.0
2102,Seguimiento,37773.0,20280326,"U303","P203","Audifonos",1,Tarjeta,Pendiente,0.0,True,37773.0
2103,Compra,38001.0,20271115,"U304","P204","Teclado",2,Sinpe,Completado,0.0,True,0.0
2104,Compra,38229.0,20270708,"U305","P205","Mouse",3,Efectivo,Cancelado,0.0,True,0.0
2105,Compra,38457.0,20270302,"U306","P206","Monitor",1,Tarjeta,Pendiente,0.0,True,0.0
2106,Apartado,38685.0,20261027,"U307","P207","Impresora",1,Sinpe,Completado,0.0,True,38685.0
2107,Visualizacion,38913.0,20260625,"U308","P208","Camara",1,Efectivo,Cancelado,0.0,True,38913.0
2108,Visualizacion,39141.0,20280223,"U309","P209","Consola",1,Tarjeta,Pendiente,0.0,True,39141.0
2109,Devolucion,39369.0,20271026,"U310","P210","Smartwatch",1,Sinpe,Completado,0.0,True,39369.0
2110,Seguimiento,39597.0,20270630,"U311","P211","Bocina",1,Efectivo,Cancelado,0.0,True,39597.0
```

Al seleccionar la opción “Salir”, el sistema finaliza su ejecución de manera ordenada y muestra un mensaje indicando que se está cerrando correctamente. En este punto, no se realizan más procesos ni cálculos, simplemente se termina la interacción con el usuario.

Es importante entender que, al salir del sistema, el archivo eventos.csv no se borra ni se modifica incorrectamente. Toda la información que se ha generado y guardado durante el uso del programa permanece intacta. Esto significa que los eventos registrados siguen almacenados tal como estaban al momento de salir.

Cuando el sistema se vuelva a ejecutar en otro momento, continuará trabajando sobre ese mismo archivo, utilizando los datos ya existentes. Los nuevos eventos que se generen se agregarán al final del archivo, manteniendo un crecimiento progresivo de la información.

Además, el sistema está diseñado para evitar inconsistencias, por lo que no se generan duplicados ni se sobrescriben datos anteriores. Cada evento nuevo tiene su propio identificador y se integra correctamente con los registros existentes.

Descripción del problema.

Actualmente los sistemas digitales dependen en gran parte a los procesamiento continuos de información generada por los usuarios. Cada interacción dentro de una plataforma como: visualizar un producto, realizar una compra, efectuar una devolución o consultar información, produce datos que son registrados en forma de eventos. Estos eventos representan unidades importantes de información que permiten controlar las acciones de los usuarios y el funcionamiento global del sistema.

El presente proyecto aborda la necesidad de diseñar e implementar un sistema en el lenguaje de programación Haskell, cuyo propósito principal es gestionar, procesar y analizar eventos generados dentro de una plataforma de comercio electrónico simulada. El sistema debe ser capaz de manejar grandes volúmenes de información de manera eficiente, aplicando técnicas propias de la programación funcional, tales como el uso de funciones puras, composición de funciones, recursividad, y el manejo de listas como estructuras principales de datos.

Cada evento dentro del sistema representa una acción específica realizada por un usuario y contiene múltiples atributos que describen completamente dicha acción. Entre estos atributos se incluyen: un identificador único del evento, la categoría de la acción (por ejemplo, compra, visualización, devolución, etc.), el valor monetario asociado a la transacción, la marca de tiempo que indica el momento exacto en el que ocurrió el evento, el identificador del usuario, el producto involucrado, la cantidad relacionada con la operación, el método de pago utilizado, el estado del evento, así como otros campos adicionales como impuestos aplicados, etiquetas de clasificación y el total calculado del evento.

Dado que el sistema simula un entorno dinámico y en constante crecimiento, los eventos no son estáticos, sino que se generan de forma continua y automática, lo que convierte el conjunto de datos en un flujo potencialmente infinito. Por esta razón, el sistema debe estar preparado para realizar operaciones de análisis en tiempo de ejecución, sin depender de almacenamiento persistente.

El problema central que se busca resolver consiste en desarrollar un conjunto de funcionalidades que permitan extraer información significativa a partir de los eventos generados, transformando datos en conocimiento útil para la toma de decisiones. Para ello, el sistema debe implementar operaciones de filtrado, transformación, agrupación y agregación de datos.

Las funcionalidades principales del sistema se organizan en distintos módulos de análisis:

Transformación de eventos

El sistema debe ser capaz de modificar los eventos aplicando reglas de negocio. Esto incluye, por ejemplo, el cálculo de impuestos sobre eventos de tipo compra, así como la identificación de eventos de alto valor en función de promedios por categoría.

Análisis de datos

El sistema realiza cálculos estadísticos sobre el conjunto de eventos. Entre estos análisis se incluyen el cálculo del monto total generado por todas las transacciones, así como el cálculo de promedios de montos agrupados por categoría y por año.

Análisis temporal

El sistema también debe permitir el análisis de la información desde una perspectiva temporal. Esto implica identificar patrones de comportamiento en distintos periodos de tiempo, como meses con mayor volumen de transacciones, eventos más antiguos y recientes, así como intervalos de tiempo con mayor actividad.

Búsqueda por rangos de fechas

Otra funcionalidad importante es la capacidad de filtrar eventos dentro de un intervalo de tiempo específico. El usuario puede definir una fecha de inicio y una fecha de fin, y el sistema debe retornar todos los eventos que se encuentren dentro de ese rango. Esto permite realizar consultas focalizadas sobre períodos concretos.

Generación de estadísticas

El sistema debe generar reportes estadísticos generales que resuman el comportamiento global de los eventos. Esto incluye la cantidad de eventos por categoría, el evento con mayor y menor valor, y el día con mayor actividad registrada. Estas estadísticas pueden ser exportadas en formatos estructurados como CSV para su análisis externo.

El sistema completo se ejecuta mediante una interfaz de consola basada en un menú interactivo, el cual permite al usuario navegar entre las diferentes funcionalidades de forma ordenada. Cada opción del menú ejecuta procesos de análisis sobre los datos existentes en memoria.

Diseño del programa:

El sistema está diseñado en Haskell bajo una arquitectura modular, donde cada módulo cumple una función específica dentro del flujo general del programa. La estructura se divide principalmente en **Types**, **Utils**, **Core**, **Services** y **UI**, lo que permite separar claramente la lógica de negocio, las utilidades reutilizables, la presentación y el manejo de datos externos como se muestra en la figura 22.

Figura 22



En el módulo **Types.Modelos** se definen las estructuras principales del sistema, como Evento, Categoría, Estado y MetodoPago. Estas estructuras permiten representar de forma estricta los datos del sistema evitando errores

comunes como el uso de cadenas inconsistentes. Evento es la entidad central, ya que contiene toda la información relevante de cada registro, incluyendo valor, cantidad, impuestos, total y metadatos como usuario, producto y fecha en formato entero como se muestra en la figura 23.

Figura 23

```
data Evento = Evento
  { idEvento      :: Int
  , categoria     :: Categoria
  , valor         :: Float
  , timestamp     :: Int
  , usuarioId     :: String
  , productoId    :: String
  , producto      :: Producto
  , cantidad      :: Int
  , metodoPago    :: MetodoPago
  , estado        :: Estado
  , impuesto      :: Float
  , etiqueta      :: Bool
  , total         :: Float
  } deriving (Show, Read, Eq)
```

En **Utils.Calculos** se implementan todas las operaciones matemáticas y de transformación de datos. Aquí se encuentran funciones como `calcularSubtotal`, `calcularImpuesto13` como se muestra en figura 24 y `redondear`, las cuales permiten mantener la lógica numérica separada del resto del sistema. También se implementan funciones como `calcularTotales`, que centraliza la lógica de cálculo dependiendo del tipo de evento, y `totalAjustado`, que permite manejar casos especiales como devoluciones. Además, se incluyen funciones de agregación como `calcularSumaTotales`, `contarEventos` y `calcularPromedioLocal`, que son utilizadas posteriormente en los módulos de análisis.

Figura 24

```
calcularImpuesto13 :: Float -> Float
calcularImpuesto13 subtotal = subtotal * 0.13
```

El módulo **Core.Impuestos** se encarga de aplicar reglas de negocio relacionadas con impuestos. La función principal `aplicarImpuestos` recorre la lista de eventos y aplica la lógica definida en `aplicarImpuestoEvento`. Esta función valida si un evento ya tiene impuesto aplicado, si es una compra o si debe mantenerse sin

cambios. En caso de ser una compra, se calcula el subtotal multiplicando valor por cantidad, luego se aplica un impuesto del 13% y finalmente se actualiza el total del evento. También se incluyen funciones auxiliares como `esCompra`, `yaTieneImpuesto` y `filtrarCompras`, que permiten segmentar correctamente los datos antes de procesarlos.

En **Core.Etiquetas** se maneja la clasificación de eventos según su valor respecto al promedio de su categoría. La función principal `etiquetarAltoValor` calcula primero los promedios por categoría utilizando `calcularPromedios` y luego etiqueta cada evento comparando su valor individual con el promedio correspondiente. Si el valor del evento es mayor al promedio de su categoría, se marca como `True` en el campo `etiqueta`, en caso contrario se marca como `False`. Esto permite identificar eventos de alto valor de forma dinámica.

El módulo **Core.AnalisisTemporal** se encarga del análisis basado en fechas. Aquí se implementan funciones como `analisisMesDia`, que determina el mes con mayor ingreso total y el día con mayor actividad. Para esto se utilizan funciones auxiliares como `mesConMayorMonto` y `diaMasActivo`. También se implementa `analisisExtremos`, que identifica el evento más antiguo y el más reciente utilizando `eventosExtremos`. Además, `analisisResumen` agrupa eventos en intervalos de tiempo definidos por el usuario, utilizando funciones como `agruparPorIntervalo` y `agruparPorDias`, las cuales segmentan los eventos según rangos de fechas.

El sistema de fechas está centralizado en **Types.Fecha**, donde se manejan conversiones entre enteros y el tipo `Day`. Se implementan funciones como `intToDay`, `dayToInt`, `extraerAnio`, `extraerMes` y `extraerDia`, que permiten manipular fechas en formato `YYYYMMDD`. También se incluyen funciones de validación como `parseFecha` y `esFechaValida`, que aseguran que los datos ingresados tengan formato correcto. Adicionalmente, se manejan utilidades para formatear fechas en texto legible y convertirlas a nombres de meses.

En **Types.Modelos** se definen las entidades principales del sistema, incluyendo `Evento`, `Categoria`, `Estado` y `MetodoPago`. `Categoria` representa el tipo de acción como `Compra` o `Devolucion`, mientras que `Estado` define el estado del

evento como Completado, Pendiente o Cancelado. Evento agrupa todos los datos necesarios para el procesamiento del sistema, incluyendo cálculos de impuestos, totales y etiquetas.

El módulo **Utils.Calculos** también incluye algoritmos más avanzados como `ordenarEventos`, agrupar eventos por fechas, separar eventos por intervalos y construir rangos temporales. Estas funciones permiten realizar análisis sobre grandes volúmenes de datos de manera eficiente. Además, se implementan funciones para obtener promedios por categoría, filtrar eventos y construir estadísticas agregadas.

En cuanto a eventos extremos, el sistema incluye funciones como `eventoMaxCSV`, `eventoMinCSV` y `eventoMinSinDevolucionesCSV`, que permiten identificar los eventos con mayor y menor impacto económico. También se incluye lógica para manejar devoluciones, ajustando los valores totales en negativo cuando corresponde.

El módulo **Utils.CSV** se encarga de la persistencia de datos. Aquí se implementa la conversión de eventos a formato CSV mediante `eventoToCSV`, así como el proceso inverso con `csvToEventoSeguro`. También se incluye validación de datos mediante `validarEvento`, que evita que se almacenen registros con valores inválidos. La función `agregarEventoSeguro` maneja la inserción segura en archivos, evitando duplicados y errores de escritura mediante manejo de excepciones con `try`. Además, se implementa lectura segura de archivos con `leerEventosSeguro`, que ignora líneas inválidas.

El módulo **Utils.Formato** se encarga del formateo visual de datos. Aquí se implementa la función `formatearMonto`, que convierte valores numéricos a formato de moneda con separadores de miles y dos decimales. También se incluyen funciones de alineación de texto como `ajustarTexto`, `ajustarNumero` y `alinearTexto`, que permiten estructurar salidas en consola. Además, se manejan funciones para obtener promedios por categoría y formateo de columnas.

En **Utils.Listas** se implementan utilidades genéricas para manejo de listas como `contarElemento`, `filtrarElemento`, `maximoPorCriterio` y `compararPorCriterio`. Estas funciones permiten reutilizar lógica de conteo, filtrado y comparación en diferentes partes del sistema sin duplicación de código.

En **Utils.Colores** se implementa todo el sistema de visualización en consola utilizando códigos ANSI. Se definen colores como rojo, verde, amarillo, azul, magenta, cyan y blanco, además de estilos como negrita. También se incluyen funciones como `okMsg`, `errorMsg`, `warningMsg` e `infoMsg`, que permiten mostrar mensajes con formato visual dependiendo del tipo de información.

Algoritmos más importantes creados

1. Algoritmo de cálculo de impuestos

Este algoritmo se encarga de aplicar impuestos únicamente a los eventos de tipo compra. Primero recorre la lista de eventos y filtra aquellos que pertenecen a la categoría compra. Luego, para cada evento válido, calcula el subtotal multiplicando el valor unitario por la cantidad. Después aplica un impuesto del 13% sobre ese subtotal. Finalmente, suma el impuesto al subtotal para obtener el total del evento actualizado. El algoritmo también incluye una validación previa para evitar recalcular impuestos en eventos que ya los tienen aplicados como se muestra en la figura 25.

Figura 25

```
aplicarImpuestoEvento :: Evento -> Evento
aplicarImpuestoEvento evento
  | yaTieneImpuesto evento =
    evento
  | esCompra evento =
    calcularImpuestoCompra evento
  | otherwise =
    evento { impuesto = 0 }
```

2. Algoritmo de promedio de compras

Este algoritmo calcula el promedio del monto total de las compras registradas en el sistema. Primero filtra todos los eventos de tipo compra. Luego suma los totales ajustados de cada evento, considerando casos especiales como devoluciones. Después cuenta la cantidad de compras existentes. Finalmente divide la suma total entre la cantidad de elementos. Si no existen compras, el algoritmo devuelve cero para evitar errores de división.

3. Algoritmo de etiquetado de eventos por valor

Este algoritmo clasifica cada evento según su valor en comparación con el promedio de su categoría. Primero calcula el promedio de cada categoría existente en la lista de eventos. Luego recorre cada evento y obtiene el promedio correspondiente a su categoría. Si el valor del evento es mayor que el promedio de su categoría se marca con etiqueta verdadera. En caso contrario, se marca como falsa. Esto permite identificar automáticamente eventos con valores altos respecto a su grupo.

4. Algoritmo de análisis del mes con mayor ingreso

Este algoritmo determina cuál mes tiene el mayor ingreso total. Primero agrupa los eventos por mes utilizando la fecha del timestamp. Luego suma los valores totales de cada grupo mensual. Después compara los resultados entre todos los meses y selecciona el que tenga el valor más alto. El resultado final es el mes junto con su total acumulado.

5. Algoritmo de día con mayor actividad

Este algoritmo identifica el día con mayor cantidad de eventos registrados. Primero convierte las fechas de los eventos a un formato legible. Luego agrupa los eventos por día. Después cuenta cuántos eventos hay en cada grupo. Finalmente selecciona el día con mayor cantidad de ocurrencias. Este proceso permite identificar los días con mayor actividad en el sistema.

6. Algoritmo de eventos extremos

Este algoritmo encuentra el evento con mayor valor y el evento con menor valor. Primero recorre toda la lista de eventos. Luego utiliza el total ajustado para comparar los valores, considerando devoluciones como valores negativos. El máximo se obtiene seleccionando el evento con mayor total ajustado, y el mínimo se obtiene de forma similar, pero filtrando casos especiales cuando es necesario. El resultado se devuelve en formato CSV.

7. Algoritmo de agrupación por intervalos de tiempo

Este algoritmo divide los eventos en grupos según intervalos de días definidos dinámicamente. Primero ordena los eventos por fecha. Luego define una fecha de inicio y una fecha final según el rango de datos. Después crea ventanas de tiempo sumando días al intervalo definido por el usuario. En cada ventana se separan los eventos que pertenecen a ese rango. Este proceso se repite hasta recorrer toda la lista de eventos.

8. Algoritmo de análisis de rango de fechas

Este algoritmo obtiene la fecha más antigua y la más reciente del sistema. Primero toma todos los eventos y extrae sus timestamps. Luego convierte estos valores a formato de fecha. Después identifica el valor mínimo y máximo. Con esto construye un rango de fechas que se utiliza para otros análisis como agrupación o estadísticas temporales.

9. Algoritmo de cálculo de totales por evento

Este algoritmo calcula el total final de cada evento dependiendo de su categoría. Primero calcula el subtotal multiplicando valor por cantidad. Luego evalúa la categoría del evento. Si es una compra, aplica un impuesto del 13% y lo suma al subtotal. Si no es compra, el total se mantiene como el subtotal. En caso de devoluciones, el total se invierte a negativo.

10. Algoritmo de lectura y escritura de CSV

Este algoritmo maneja la persistencia de datos. Para guardar, convierte cada evento a una cadena CSV concatenando todos sus campos separados por comas.

Antes de guardar, valida que el evento sea correcto y que no exista duplicado en el archivo. Para leer, abre el archivo, separa cada línea, convierte cada línea a un evento y descarta aquellas que no sean válidas. Esto garantiza integridad en los datos.

Librerías usadas:

Librería	Definición	Uso	Justificación
System.IO	Librería para entrada y salida de archivos	Leer y escribir archivos CSV (readFile, writeFile, appendFile)	Esta librería es importante en el sistema porque permite la interacción directa con archivos, lo cual es la base de la persistencia de eventos en formato CSV. Se utiliza principalmente en funciones como leerEventosSeguro, donde se usa readFile para cargar los eventos almacenados, en guardarEventos donde se usa writeFile para sobrescribir toda la información del sistema, y en agregarEventoSeguro donde se emplea appendFile para añadir nuevos eventos sin perder los anteriores. La razón de su uso es que el sistema necesita almacenar y recuperar datos de manera persistente, simulando un comportamiento similar a una base de datos, pero de forma sencilla mediante archivos de texto. Sin esta librería no sería posible manejar la entrada y salida de datos desde el sistema de archivos.
System.Directory	Permite interactuar con el sistema de archivos	Verificar si un archivo existe (doesFileExist)	Esta librería se utiliza para verificar la existencia de archivos antes de intentar leerlos, evitando errores en tiempo de ejecución se aplica en la función leerEventosSeguro, específicamente con doesFileExist, lo cual permite comprobar si el archivo CSV existe antes de intentar abrirlo. Su uso es importante porque el sistema no puede asumir que siempre habrá datos almacenados previamente, ya que los eventos pueden no existir en la primera ejecución. Gracias a esta librería

			se mejora la robustez del sistema y se evita que el programa falle por intentar leer un archivo inexistente.
Control.Exception	Manejo de excepciones y errores en ejecución	Captura de errores al escribir archivos con try	Se utiliza para el manejo de errores durante operaciones de entrada y salida, especialmente en el manejo de archivos. En este proyecto se aplica en la función agregarEventoSeguro, donde se usa try para capturar posibles fallos al escribir en el archivo con appendFile. La justificación de su uso es que las operaciones con archivos pueden fallar por múltiples razones (permisos, archivos bloqueados, errores del sistema), por lo que es necesario controlar estas excepciones para evitar que el programa se cierre inesperadamente.
Data.Time	Manejo de fechas y tiempo	Conversión de fechas, comparación, suma de días (Day, addDays, toGregorian)	Esta librería es utilizada para el manejo de fechas y tiempo dentro del sistema, especialmente en la generación y análisis de eventos por timestamp. Aunque en el código mostrado no se detalla su uso completo, su función principal dentro del proyecto es permitir representar tiempos de los eventos, realizar conversiones de fechas, manipular intervalos de tiempo y realizar análisis temporales como eventos más antiguos o recientes. Se usa en funciones relacionadas con análisis temporal y generación de eventos dinámicos se justifica porque el sistema simula eventos reales donde el tiempo es un atributo clave para el análisis de comportamiento del usuario.
	Funciones para manipular listas	Filtrar, ordenar, agrupar, eliminar duplicados	Es una de las librerías más importantes del proyecto, ya que permite manipular listas de eventos, que es la estructura central del sistema. Se utiliza en múltiples funciones como leerEventosSeguro (con catMaybes indirectamente apoyado en filtrado), en contarElemento,

Data.List		(filter, map, sort, nub, maximumBy)	filtrarElemento, maximoPorCriterio, y en procesos de análisis como filtrado de eventos, agrupación por categorías y cálculo de estadísticas. Funciones como filter, map, sort, maximumBy o nub son esenciales para transformar y analizar los eventos el paradigma funcional de Haskell se basa en el manejo de listas, y esta librería proporciona las herramientas necesarias para realizar operaciones complejas de forma declarativa y eficiente.
Data.Ord	Comparación de valores	Comparar eventos por criterios personalizados (comparing, on)	Se utiliza para realizar comparaciones avanzadas entre eventos según criterios personalizados. En este proyecto se aplica indirectamente en funciones como maximoPorCriterio, donde se necesita comparar eventos según un valor derivado (por ejemplo, monto o timestamp). Funciones como comparing permiten definir reglas de ordenamiento sin escribir lógica manual compleja. Su uso es importante porque el sistema necesita encontrar eventos máximos o mínimos según diferentes atributos, lo cual es clave en el módulo de estadísticas.
Data.Function	Funciones de apoyo para programación funcional	Uso de on para comparaciones	Esta librería se utiliza principalmente para la función on, que permite simplificar comparaciones basadas en una transformación previa de los datos. En este proyecto se usa en conjunto con Data.Ord para realizar comparaciones más limpias y expresivas en funciones como ordenamientos o búsquedas de máximos. La justificación de su uso es que mejora la legibilidad del código funcional y evita escribir comparaciones repetitivas, permitiendo enfocarse en la lógica
	Conversión de texto a	Convertir strings a	Se utiliza para convertir cadenas de texto en otros tipos de datos, especialmente números. En este proyecto se usa principalmente en la función csvToEventoSeguro,

Text.Read	otros tipos	números (read)	donde se convierte cada campo del CSV usando read para reconstruir un Evento desde texto. Su importancia radica en que el archivo CSV almacena todo como texto, pero el sistema necesita reconstruir estructuras tipadas correctamente. Sin esta librería no sería posible transformar los datos leídos en estructuras utilizables dentro del programa.
Numeric	Formateo numérico	Mostrar decimales (showFFloat)	Se utiliza para el formateo de valores numéricos, específicamente en la presentación de montos. En este proyecto se aplica en la función formatearMonto, donde se usa showFFloat para limitar los decimales de los valores monetarios. La justificación de su uso es mejorar la presentación de los datos al usuario, asegurando que los montos se muestren de forma clara y profesional (por ejemplo, con dos decimales), lo cual es importante en un sistema de comercio electrónico donde los valores financieros deben ser precisos y legibles.

Análisis de resultados

Objetivos alcanzados

- Implementación de un sistema funcional en Haskell para el registro y procesamiento de eventos de una plataforma de comercio electrónico, modelando cada evento como un dato algebraico con atributos como categoría, valor, timestamp, usuario y producto.
- Desarrollo de la capacidad de generación y manipulación de eventos en memoria, permitiendo simular el comportamiento de un sistema que recibe información de forma continua.
- Implementación de transformación de eventos mediante funciones puras, aplicando impuesto del 13% a eventos de tipo compra y etiquetando eventos de alto valor según el promedio de su categoría.

- Desarrollo de análisis de datos mediante operaciones funcionales como filtrado, mapeo y reducción, permitiendo calcular el monto total de los eventos y el promedio por categoría y año.
- Implementación de análisis temporal, permitiendo identificar el mes con mayor monto, el día más activo, así como el evento más antiguo y el más reciente del sistema.
- Desarrollo de búsqueda de eventos por rango de fechas, facilitando la segmentación de información según intervalos temporales específicos.
- Implementación de un módulo de estadísticas generales que incluye conteo de eventos por categoría, evento con mayor y menor monto, y día con mayor actividad.
- Generación de reportes en formato CSV para exportación de resultados, cumpliendo parcialmente con el requisito de exportación de estadísticas.
- Uso de programación funcional en Haskell, aplicando conceptos como inmutabilidad, funciones de orden superior, composición de funciones y procesamiento de listas como estructura principal del sistema.
- Implementación de lectura y escritura de archivos CSV, permitiendo persistencia básica de datos y carga de información desde archivos externos.
- Organización del sistema en módulos independientes, separando lógica en componentes como Core, Utils y Types, lo que mejora la mantenibilidad, claridad y escalabilidad del proyecto.
- Implementación completa de exportación de estadísticas en CSV.
- Simulación de flujo continuo o infinito de eventos en tiempo real.
- Implementación de persistencia de datos mediante archivos CSV, permitiendo almacenar y recuperar eventos desde almacenamiento físico. Esto garantiza que la información no se pierda al cerrar el sistema y pueda reutilizarse en ejecuciones posteriores. Además, se implementó un mecanismo de validación para evitar la inserción de eventos duplicados, verificando la existencia previa del identificador antes de realizar el guardado.

Justificación de toma de decisiones

En este sistema se tomaron decisiones de diseño basadas principalmente en el uso de funciones de orden superior, ya que el procesamiento de datos se realiza sobre listas de eventos y era necesario manipular grandes volúmenes de información de forma clara, modular y sin recurrir a estructuras imperativas como bucles explícitos. Esto permite expresar la lógica en términos de transformaciones, filtrados, agrupaciones y reducciones, lo que hace el código más declarativo y fácil de mantener.

Map

La primera función de orden superior utilizada es **map**. Esta función recibe otra función como parámetro y la aplica a cada elemento de una lista. En el sistema se utiliza, por ejemplo, en la función `calcularMontoTotal`, donde se aplica `map totalAjustado eventos`. En este caso, `map` recorre todos los eventos y aplica la función `totalAjustado` a cada uno, generando una nueva lista con los totales calculados. Luego, esa lista es reducida con `sum`. También se utiliza en otros puntos como `map totalAjustado eventosActualizados`, lo que permite transformar todos los eventos de manera uniforme sin necesidad de iteraciones manuales.

Filter

La segunda función es `filter`, que permite seleccionar únicamente los elementos que cumplen una condición lógica. En el sistema se usa en múltiples lugares, por ejemplo, en `filtrarEventosPorCategoria`, donde se escribe `filter (\e -> categoria e == catBuscada) eventos`. Aquí `filter` recorre la lista de eventos y solo conserva aquellos cuya categoría coincide con la buscada. También se usa en el módulo de búsqueda por fechas, donde `filter` se utiliza indirectamente en `filtrarEventosEnRango` para quedarse únicamente con eventos cuyo timestamp está entre una fecha inicial y una final.

Any

La tercera función es `any`, que se utiliza para verificar si al menos un elemento de una lista cumple una condición. En el sistema se emplea en funciones de validación o búsqueda, como cuando se verifica si existe un evento con un

identificador específico: `any (\e -> idEvento e == idBuscado)`. Esta función recorre la lista de eventos y devuelve `True` si encuentra al menos uno que cumpla la condición, o `False` en caso contrario. Esto evita tener que recorrer manualmente toda la lista con lógica adicional.

maximumBy y minimumBy

La cuarta familia de funciones son `maximumBy` y `minimumBy`, que permiten obtener el elemento máximo o mínimo de una lista usando una función de comparación personalizada. En el sistema se usan en funciones como `eventoMaxCSV` y `eventoMinSinDevolucionesCSV`. Por ejemplo, `maximumBy (compare \on`totalAjustado)` `eventosselecciona` el evento con mayor valor según el total ajustado. Aquí, `on totalAjustado`` define el criterio de comparación, lo que permite que la selección no dependa del evento en sí, sino de un valor calculado a partir de él. Esto también se utiliza para encontrar eventos extremos en estadísticas.

mapM y mapM_

La quinta función de orden superior es `mapM` y `mapM_`, que son versiones de `map` diseñadas para trabajar dentro del contexto de IO. En el sistema se utilizan para ejecutar acciones con efectos secundarios sobre listas. Por ejemplo, en la generación de eventos se usa `mapM (crearEvento fechaBase)` `indices`, lo que permite crear múltiples eventos dentro del contexto de entrada/salida. En la interfaz de usuario se utiliza `mapM_ imprimirFila eventosOrdenados`, donde cada evento se imprime en pantalla sin devolver resultados, solo ejecutando la acción.

group y groupBy

La sexta función es `group` y `groupBy`, que permiten agrupar elementos de una lista. En el sistema se usan en el módulo de estadísticas, específicamente en `contarEventosPorCategoria`, donde primero se ordenan los eventos por categoría y luego se aplica `groupBy ((==) \on`categoria)`. Esto agrupa eventos con la misma categoría en sublistas, lo que permite posteriormente contarlos fácilmente. También se utiliza `group`` en el cálculo de días con mayor actividad, donde se agrupan timestamps repetidos para contar ocurrencias.

sort, sortBy y sortOn

La séptima familia de funciones es `sort`, `sortBy` y `sortOn`, que permiten ordenar listas según distintos criterios. En el sistema se utiliza `sortOn timestamp` en la función `ordenarEventos`, lo que ordena los eventos de forma ascendente por fecha. También se utiliza `sortBy (compare \on` categoria)` en procesos donde se requiere agrupar correctamente antes de aplicar `groupBy``. Estas funciones aseguran que los datos estén organizados antes de ser procesados o mostrados, especialmente en análisis temporales.

lookup

La octava función es `lookup`, que permite buscar un valor asociado a una clave dentro de una lista de pares. En el sistema se usa en `obtenerPromedioCategoria` como en la figura 26, donde se busca el promedio asociado a una categoría específica dentro de una lista de tuplas (`Categoria`, `Float`). Si la categoría existe, se devuelve su valor; si no, se retorna `Nothing`. Esta función simplifica la búsqueda en estructuras tipo diccionario sin necesidad de recorrer manualmente la lista.

Figura26

```
obtenerPromedioCategoria :: [(Categoria, Float)] -> Categoria -> Maybe F
✓ obtenerPromedioCategoria promedios catBuscada =
  lookup catBuscada promedios
```

Encadenamiento de funciones

En cuanto al encadenamiento de funciones, el sistema se diseñó para que las salidas de unas funciones sean las entradas de otras, formando cadenas de procesamiento de datos. Por ejemplo, en el cálculo de totales se encadenan `map` y `sum`, donde primero se transforma cada evento con `map totalAjustado eventos` y luego se reduce la lista con `sum`. En la búsqueda por rango de fechas se encadenan `filter` y `sortBy`, primero filtrando los eventos dentro del rango y luego ordenándolos por fecha antes de mostrarlos con `mapM_ imprimirFila`. En el sistema de estadísticas se encadenan `sortBy`, `groupBy` y `map`, donde primero se ordena la lista,

luego se agrupan los eventos por categoría y finalmente se cuentan los elementos de cada grupo como se muestra en la figura 27.

Figura 27

```
contarEventosPorCategoría :: [Evento] -> [(String, Int)]
contarEventosPorCategoría eventos =
  let ordenar = sortBy (compare `on` categoría) eventos
      agrupar = groupBy ((==) `on` categoría) ordenar
      contar grupo = (show (categoría (head grupo)), length grupo)
  in map contar agrupar
```

Bitácora

Link:

<https://github.com/Alice-Arias/PY2-Sistema-de-eventos-comerciales-/commits/main/>